

Relatório Técnico — Aula 8

Exploração de Armazenamento para o Mini Radar ENEM

Disciplina: Computação em Nuvem **Atividade:** Aula 8 — Laboratório exploratório de persistência e armazenamento **Projeto base:** Mini Radar ENEM (aplicação Flask containerizada — ver `aula_07_producao/`) **Data de execução:** 02/10/2026

Nota técnica sobre o ambiente

Os experimentos foram executados em um ambiente de laboratório cujo runtime de containers é o **Podman 5.2.3** operado através da interface de linha de comando compatível com `docker` (`docker version` reporta `Podman Engine / Version 5.2.3`). Todos os comandos do roteiro foram executados sem alteração de sintaxe — Podman implementa a mesma CLI de volumes, bind mounts e `run/exec/rm` usada pelo Docker. Duas adaptações pontuais, sempre preservando o resultado e a evidência, estão documentadas na seção de cada desafio:

1. O passo interativo `docker run -it` (2.3) foi substituído por `docker run ... sh -c '...'`, pois o ambiente é não-interativo (sem TTY). O efeito — gravar e ler um arquivo no volume dentro do container — é idêntico.
2. O utilitário `time` não existe no shell do `ubuntu:22.04` minimal; a medição de tempo do Desafio 7 foi feita com o `time` do shell do host, que mede o tempo real de ponta a ponta da operação (incluindo o start do container).

Durante a execução, o Podman apresentou ocasionalmente o erro `would cause a deadlock; please run 'podman system rename'` (esgotamento transitório do pool de locks ao encadear muitos containers `--rm`). A correção recomendada pelo próprio runtime (`docker system rename`) foi aplicada antes de cada bloco. Isso não afeta os resultados nem as conclusões — é uma particularidade operacional do ambiente, registrada aqui por transparência.

1. Preparação do ambiente

```
$ docker --version
docker version 5.2.3    # (Podman Engine 5.2.3, CLI compatível com docker)

$ docker run --rm hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

O runtime está funcional e capaz de puxar imagens do registry e executar containers.

2. Desafio obrigatório — Persistência com Docker Volume

Objetivo: demonstrar que os dados sobrevivem à remoção do container quando armazenados em um Docker Volume.

2.1 / 2.2 — Criar e conferir o volume

```
$ docker volume create radar-dados
radar-dados
```

```
$ docker volume ls
DRIVER    VOLUME NAME
local     radar-dados
```

2.3 — Criar container, montar o volume e gravar

Adaptação: `-it` `sh -c` (ambiente sem TTY). Mesmo efeito.

```
$ docker run --name radar-storage -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'echo "Radar ENEM - Aula 8" > /dados/versao.txt && cat /dados/versao.txt'
Radar ENEM - Aula 8
```

2.4 — Remover o container

```
$ docker rm radar-storage
radar-storage
```

2.5 — Criar OUTRO container e ler o mesmo arquivo

```
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 cat /dados/versao.txt
Radar ENEM - Aula 8
```

Resultado esperado confirmado: o texto continua disponível mesmo depois da remoção do primeiro container.

2.6 — Inspecionar o volume

```
$ docker volume inspect radar-dados
[
  {
    "Name": "radar-dados",
    "Driver": "local",
    "Mountpoint": "/var/lib/containers/storage/volumes/radar-dados/_data",
    "CreatedAt": "2026-10-02T13:01:15.44879721Z",
    "Scope": "local",
    "MountCount": 0
  }
]
```

Registro / análise do obrigatório

- **O que aconteceu com o container?** O container `radar-storage` foi removido (`docker rm`). Seu sistema de arquivos de escrita (a camada *writable* efêmera) deixou de existir.
- **O que aconteceu com o arquivo?** O arquivo `versao.txt` permaneceu intacto porque foi gravado no diretório montado `/dados`, que aponta para o volume `radar-dados` — um recurso **externo** ao container, armazenado em `/var/lib/containers/storage/volumes/radar-dados/_data` no host.
- **Diferença entre o ciclo de vida do container e o do dado:** o container é **efêmero e descartável** (criado, parado e removido a qualquer momento); o volume tem **ciclo de vida independente**, sobrevivendo à destruição de qualquer container que o utilize, até ser removido explicitamente com `docker volume rm`. Essa separação é o fundamento de persistência em aplicações containerizadas.

3. Desafios explorados

Foram realizados **8 desafios** (muito além do mínimo de 3): 1, 2, 3, 4, 5, 7, 8 e 9 com execução real, e o 6 (Object Storage) como proposta conceitual, conforme autorizado pelo roteiro na ausência de um serviço S3 no laboratório.

Desafio 1 — Filesystem × Volume

```
# (a) Container SEM volume grava em /tmp
$ docker run --rm ubuntu:22.04 sh -c 'echo "temporario" > /tmp/teste.txt && cat /tmp/teste.txt'
temporario

# (b) Container NOVO sem volume: o arquivo sumiu
$ docker run --rm ubuntu:22.04 sh -c 'cat /tmp/teste.txt || echo "ARQUIVO NAO ENCONTRADO (efemero)'"
cat: /tmp/teste.txt: No such file or directory
>>> ARQUIVO NAO ENCONTRADO (efemero)

# (c) Container COM volume grava em /dados
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'echo "persistente" > /dados/teste.txt && cat /dados/teste.txt'
persistente

# (d) Container NOVO com volume: o arquivo persiste
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 cat /dados/teste.txt
persistente
```

Descoberta: o conteúdo escrito no filesystem do container (`/tmp`) desaparece junto com o container; o conteúdo escrito no volume (`/dados`) sobrevive. É a demonstração mais direta de por que a camada de escrita do container **não** deve ser usada como persistência.

Desafio 2 — Backup e restauração

```
# Criar dado no volume
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 sh -c 'echo "backup-aula-8" > /dados/backup.txt'

# Copiar do volume para o host (bind mount ./backup-radar -> /backup)
$ docker run --rm -v radar-dados:/dados -v "$(pwd)/.../backup-radar:/backup" ubuntu:22.04 \
```

```

cp /dados/backup.txt /backup/
$ cat ./backup-radar/backup.txt
backup-aula-8

# Simular PERDA do dado no volume
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 sh -c 'rm /dados/backup.txt && cat /dados/backup.txt'
cat: /dados/backup.txt: No such file or directory
>>> arquivo removido do volume

# RESTAURAR a partir do backup salvo no host
$ docker run --rm -v radar-dados:/dados -v "$(pwd)/.../backup-radar:/backup" ubuntu:22.04 \
  sh -c 'cp /backup/backup.txt /dados/ && cat /dados/backup.txt'
restaurado:
backup-aula-8

```

Descoberta: o ciclo completo **backup perda restauração** funcionou. Um bind mount para um diretório do host é uma forma simples de extrair uma cópia de segurança para fora do volume. O arquivo de backup gerado está versionado no repositório em `backup-radar/backup.txt`.

Desafio 3 — Versionamento dos dados

```

$ docker run --rm -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'echo "versao 1" > /dados/resultados_v1.txt && echo "versao 2" > /dados/resultados_v2.txt'

$ docker run --rm -v radar-dados:/dados ubuntu:22.04 ls -lh /dados
-rw-r--r-- 1 root root 9 ... resultados_v1.txt
-rw-r--r-- 1 root root 9 ... resultados_v2.txt

```

Quando versionar é útil × quando pesa: manter versões (`_v1`, `_v2`, ...) é útil para **auditoria, reprodutibilidade e rollback** de resultados do Radar ENEM — por exemplo, poder provar qual conjunto de notas gerou um relatório publicado. O custo aumenta quando: (a) cada versão é grande (datasets), multiplicando o armazenamento; (b) o número de versões cresce sem política de expurgo; (c) a nomenclatura manual (`_vN`) vira fonte de erro. Para esses casos, versionamento **nativo de object storage** (S3 versioning) ou um esquema datado com retenção automática é preferível a arquivos manuais.

Desafio 4 — Compartilhamento entre containers (produtor/consumidor)

```

# Produtor grava
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'echo "gerado-pelo-produtor" > /dados/compartilhado.txt'

# Consumidor lê
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 cat /dados/compartilhado.txt
gerado-pelo-produtor

```

Descoberta: o mesmo volume montado em dois containers distintos permite o padrão produtor/consumidor — um serviço (ex.: processador de notas) grava e outro (ex.: API de consulta) lê, sem acoplamento direto entre eles.

Desafio 5 — Múltiplas instâncias (estado compartilhado)

```

$ docker run -d --name radar-a -v radar-dados:/dados ubuntu:22.04 sleep 300
$ docker run -d --name radar-b -v radar-dados:/dados ubuntu:22.04 sleep 300

```

```
$ docker ps
NAMES      STATUS  COMMAND
radar-a    Up ...  sleep 300
radar-b    Up ...  sleep 300

# Gravar pela A, ler pela B (simultaneamente ativos)
$ docker exec radar-a sh -c 'echo "dado-da-instancia-A" > /dados/instancia.txt'
$ docker exec radar-b cat /dados/instancia.txt
dado-da-instancia-A

$ docker rm -f radar-a radar-b
```

Risco registrado: com duas instâncias **ativas ao mesmo tempo** compartilhando o volume, uma escrita feita por A é imediatamente visível por B. Isso é poderoso, mas abre risco de **condições de corrida**: se A e B escreverem no mesmo arquivo simultaneamente, pode haver *last-write-wins*, corrupção parcial ou leitura de estado inconsistente. Volume compartilhado não oferece controle de concorrência — isso precisa vir da aplicação (locks, fila) ou de um serviço com transações (um banco de dados).

Desafio 6 — Object Storage (proposta conceitual)

```
$ docker ps
CONTAINER ID  IMAGE  COMMAND  ...  NAMES
(vazio — nenhum serviço S3-compatible rodando)
```

Não há serviço S3 (MinIO/LocalStack) disponível no laboratório; conforme a orientação do roteiro ("Não invente execução se o serviço não estiver disponível"), apresenta-se a proposta conceitual:

```
Aplicação Radar ENEM
> API/SDK S3 (ex.: boto3)
  > Bucket de objetos (ex.: s3://radar-enem/)
    resultados/exports/ (CSV/PDF gerados)
    datasets/           (microdados ENEM, arquivos grandes)
    backups/            (snapshots do estado operacional)
```

Candidatos naturais a object storage no Radar ENEM: exports de resultados (CSV/PDF), datasets de microdados do ENEM (arquivos grandes, imutáveis), imagens/relatórios e backups. Vantagens: escalabilidade praticamente ilimitada, baixo custo por GB, durabilidade alta, versionamento e políticas de ciclo de vida nativas, acesso via HTTP(S)/URLs assinadas.

Desafio 7 — Tamanho e desempenho

Tempo medido com o `time` do host (ponta a ponta, inclui start do container).

Tamanho	Taxa reportada pelo 	Tempo real (host)
10 MB	2.1 GB/s	1.382 s
50 MB	2.3 GB/s	1.373 s
100 MB	2.5 GB/s	1.501 s

```
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'dd if=/dev/zero of=/dados/teste-10mb.bin bs=1M count=10 && ls -lh /dados/teste-10mb.bin'
```

```
10485760 bytes (10 MB) copied, 0.00502 s, 2.1 GB/s
```

```
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 du -sh /dados
161M /dados # com os três arquivos .bin presentes
```

Análise e limitações do teste: a escrita em si é muito rápida (volume local em disco rápido; `/dev/zero` é altamente compressível e sequencial). O tempo real por operação (~1,4 s) é dominado pelo **overhead de criar e destruir o container**, não pela I/O — por isso os três tamanhos têm tempo real parecido mesmo com volume de dados 10× maior. O teste **não** representa cargas reais: não há aleatoriedade dos dados, não há leitura concorrente, nem latência de rede. Para avaliar desempenho de produção seriam necessárias ferramentas como `fio`, dados realistas e cenários de concorrência. Mesmo assim, a conclusão prática vale: **mover dezenas/centenas de MB tem custo** e deve pesar na escolha entre copiar arquivos entre volumes versus referenciá-los em object storage. (Os arquivos `.bin` foram removidos ao final para não poluir o repositório.)

Desafio 8 — Retenção

```
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 find /dados -maxdepth 1 -type f -printf '%f\n'
backup.txt
compartilhado.txt
instancia.txt
resultados_v1.txt
resultados_v2.txt
teste.txt
versao.txt
```

Política de retenção proposta e distinções:

- **Retenção** = por quanto tempo um dado é mantido disponível antes de ser expurgado.
- **Backup** = cópia de segurança para recuperação após falha/perda (não é o dado "vivo").
- **Exclusão** = remoção definitiva ao fim da retenção.

Classe de dado	Exemplo no volume	Retenção sugerida
Resultados atuais	<code>resultados_v2.txt</code>	Longa (enquanto for referência oficial)
Logs/temporários	<code>teste.txt</code> , <code>instancia.txt</code>	Curta (dias)
Backups	<code>backup.txt</code>	Definida por política (ex.: 30–90 dias)

Quem define a política: uma decisão de **governança** (negócio + segurança/compliance), não só técnica — considerando valor do dado, obrigações legais (ex.: dados pessoais de candidatos) e custo.

Desafio 9 — Segurança e permissões

```
# Permissões padrão (tudo root:root, 644)
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 ls -lah /dados
-rw-r--r-- 1 root root ... versao.txt

# Criar arquivo restrito (600)
$ docker run --rm -v radar-dados:/dados ubuntu:22.04 \
  sh -c 'echo "dado-restrito" > /dados/restrito.txt && chmod 600 /dados/restrito.txt && ls -lah /dados/restrito.txt'
-rw----- 1 root root 14 ... /dados/restrito.txt
```

```
# PROVA do controle: usuário NÃO-root (1000:1000) tenta ler o 600
$ docker run --rm -u 1000:1000 -v radar-dados:/dados ubuntu:22.04 cat /dados/restrito.txt
cat: /dados/restrito.txt: Permission denied # >>> ACESSO NEGADO (esperado)

# O mesmo usuário NÃO-root lê um arquivo 644 (público) normalmente
$ docker run --rm -u 1000:1000 -v radar-dados:/dados ubuntu:22.04 cat /dados/versao.txt
Radar ENEM - Aula 8
```

Descoberta (evidência forte): ao rodar o container como usuário não-root (`-u 1000:1000`), o arquivo `600` de propriedade do root ficou **inacessível** (`Permission denied`), enquanto o arquivo `644` continuou legível. Isso demonstra na prática o **princípio do menor privilégio** e a **separação entre dados administrativos (restritos) e públicos** — a aplicação deveria rodar como usuário não-root e só os dados que ela precisa ler/escrever deveriam estar acessíveis a ela.

4. Registro dos experimentos

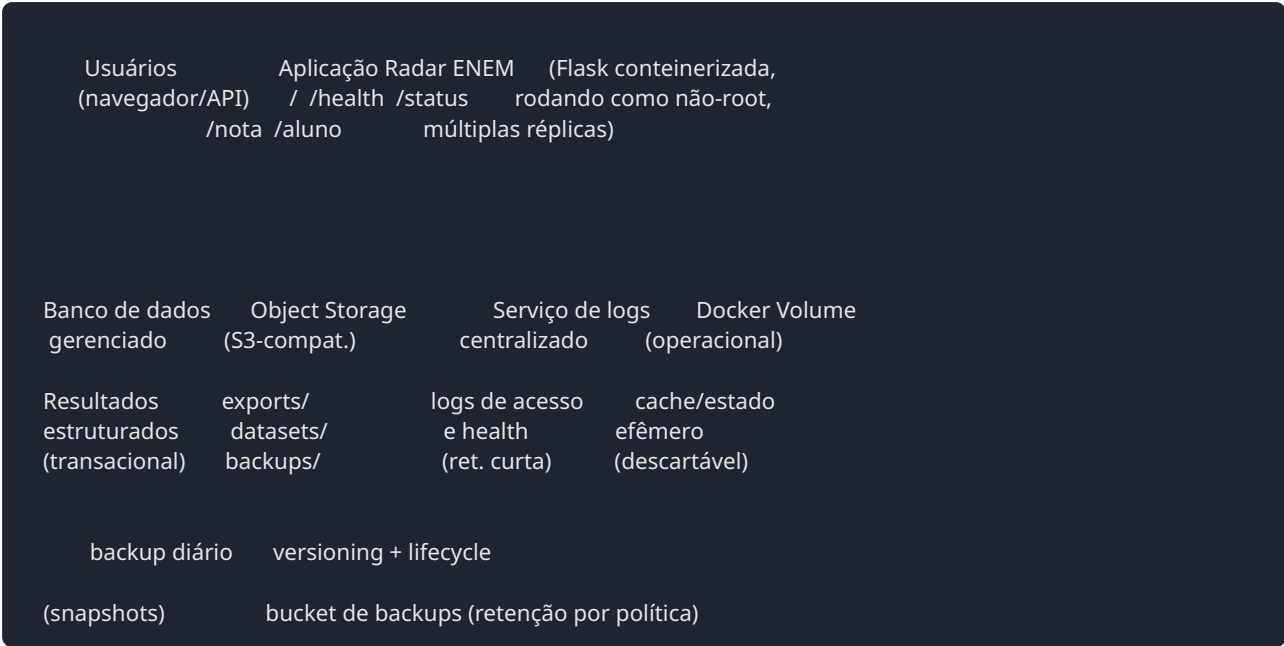
Desafio	Realizado?	Principal descoberta	Evidência
Obrigatório — Persistência	Sim	Dado no volume sobrevive à remoção do container	<code>versao.txt</code> lido por container novo (§2.5)
1 — Filesystem × Volume	Sim	<code>/tmp</code> é efêmero; <code>/dados</code> (volume) persiste	§Desafio 1 (a-d)
2 — Backup e restauração	Sim	Ciclo backup perda restauração funcional	<code>backup-radar/backup.txt</code> + §Desafio 2
3 — Versionamento	Sim	<code>_v1</code> / <code>_v2</code> úteis p/ auditoria; custo cresce	<code>ls -lh /dados</code> §Desafio 3
4 — Compartilhamento	Sim	Produtor grava, consumidor lê no mesmo volume	§Desafio 4
5 — Múltiplas instâncias	Sim	Estado compartilhado em tempo real; risco de corrida	<code>docker ps</code> + <code>exec A B</code> §Desafio 5
6 — Object Storage	Conceitual	Sem serviço S3 no lab; proposta de buckets	<code>docker ps</code> vazio + diagrama §Desafio 6
7 — Desempenho	Sim	I/O rápida; tempo dominado pelo overhead do container	Tabela 10/50/100 MB §Desafio 7
8 — Retenção	Sim	Retenção ≠/backup ≠/exclusão; é governança	<code>find /dados</code> §Desafio 8
9 — Segurança	Sim	Não-root bloqueado em arquivo 600 (menor privilégio)	<code>Permission denied</code> §Desafio 9
10 — Arquitetura	Sim	Ver seção 5	Diagrama + tabela §5

5. Arquitetura final do Radar ENEM

Com base nos experimentos, cada tipo de dado é mapeado ao modelo de armazenamento mais adequado.

Tipo de dado	Modelo	Justificativa	Acesso	Retenção/backup
Resultados estruturados (notas, classificações, consultas de /nota/ <n>)	Banco de dados gerenciado (ex.: PostgreSQL/RDS) — <i>block storage por baixo</i>	Consultas, integridade transacional e concorrência controlada (resolve o risco do Desafio 5)	Alta frequência, leitura+escrita, baixa latência	Backup diário automatizado; retenção longa (dado oficial)
Arquivos exportados (CSV/ PDF de relatórios)	Object storage (S3-compatible)	Imutáveis, servidos por URL, baratos, escaláveis	Frequência média, somente leitura após gerado	Lifecycle policy (ex.: 90 dias tier frio)
Backups	Object storage com versioning	Durabilidade alta, versionamento nativo, isolado do dado vivo	Baixa frequência (restauração sob demanda)	Retenção por política (ex.: 30–90 dias)
Logs (acesso, health, /status)	Serviço de logs / object storage (append)	Volume alto, valor cai com o tempo, raramente relido	Escrita contínua, leitura eventual	Retenção curta (ex.: 7–30 dias)
Datasets/arquivos grandes (microdados ENEM)	Object storage	Grandes, imutáveis, caros em block storage; §Desafio 7 mostra o custo de mover MBs	Leitura esporádica em lote	Retenção longa; versionamento por lançamento
Dados operacionais efêmeros (cache, estado de execução)	Docker Volume / volume gerenciado	Persistência simples entre reinícios, sem overhead de serviço	Local ao serviço	Curta; descartável

Diagrama da arquitetura proposta



Princípios aplicados vindos dos experimentos: - Separação **ciclo de vida do container** × **ciclo de vida do dado** (obrigatório + Desafio 1). - Concorrência controlada por banco, não por volume

compartilhado (Desafio 5). - Object storage para o que é grande, imutável e barato de servir (Desafios 6 e 7). - Retenção diferenciada por classe de dado (Desafio 8). - Menor privilégio e separação público/restrito (Desafio 9).

6. Respostas — relatório técnico

1. Por que o filesystem do container não deve ser tratado como persistência da aplicação?

Porque a camada de escrita do container é efêmera e vive junto com o container: ao removê-lo (`docker rm`), tudo que foi escrito fora de um volume desaparece (comprovado no Desafio 1, onde `/tmp/teste.txt` sumiu no container seguinte). Containers são projetados para serem descartáveis, recriados e escalados; dados dependentes do seu ciclo de vida seriam perdidos a cada atualização/reinício.

2. Qual evidência mostrou mais claramente que o Docker Volume preservou os dados? O

passo 2.5: após gravar `versao.txt` no volume e **remover** o container `radar-storage` (`docker rm`), um **container totalmente novo** leu o mesmo arquivo e retornou `Radar ENEM - Aula 8`. O `docker volume inspect` reforça, mostrando que o dado reside em `/var/lib/containers/storage/volumes/radar-dados/_data`, fora de qualquer container.

3. Em quais situações você utilizaria object storage, file storage ou block storage? - Object

storage: arquivos grandes, imutáveis, acessados por URL e que precisam escalar barato — exports, datasets de microdados, backups, imagens. - **File storage (volume/NFS compartilhado):** quando múltiplos processos precisam de um sistema de arquivos POSIX compartilhado (dados operacionais, área de troca produtor/consumidor). - **Block storage:** quando se precisa de I/O de baixa latência e um filesystem dedicado de alto desempenho — tipicamente **sob um banco de dados** (os resultados estruturados do Radar ENEM).

4. Que dados do Radar ENEM precisam de persistência? Resultados estruturados (notas, classificações), datasets de microdados, exports/relatórios gerados e backups. São dados que precisam sobreviver a reinícios, atualizações e à recriação de containers.

5. Que dados poderiam ser temporários? Cache, estado de execução intermediário, respostas efêmeras e logs de curta duração (como os `teste.txt` / `instancia.txt` do laboratório). Podem viver no filesystem do container ou em volume com retenção curta, pois sua perda não compromete a aplicação.

6. Qual estratégia de backup você adotaria? Backups automáticos e agendados por classe de dado: snapshots diários do banco (resultados estruturados) e cópia dos exports/datasets para um bucket de object storage **com versioning**, isolado do dado vivo. Validação periódica de restauração (o Desafio 2 provou o ciclo backup perda restauração) e retenção definida por política.

7. Como retenção e versionamento podem afetar custo e operação? Mais versões e retenções longas aumentam o volume armazenado e, portanto, o **custo** — ampliado quando os dados são grandes (Desafio 7 mostrou o peso de mover MBs). Operacionalmente, muitas versões sem política de expurgo dificultam encontrar o dado correto e aumentam a superfície de erro. A mitigação é retenção diferenciada por classe (Desafio 8), expurgo automático e versionamento nativo (ex.: S3) em vez de cópias manuais `_vN`.

8. Quais riscos aparecem quando vários serviços acessam o mesmo armazenamento?

Condições de corrida e escritas concorrentes no mesmo arquivo (comprovável no Desafio 5),

levando a *last-write-wins*, corrupção parcial ou leituras inconsistentes; além de acoplamento indesejado e falta de controle de acesso granular. Volume compartilhado não oferece transações — por isso dados críticos devem ficar em um banco com controle de concorrência.

9. Quais controles de segurança deveriam existir? Menor privilégio (aplicação rodando como usuário não-root — Desafio 9 provou o bloqueio a arquivo `600`), permissões adequadas separando dados públicos de administrativos, criptografia em repouso e em trânsito, credenciais fora do código (variáveis de ambiente/secrets, como o `.env.example` já sugere no projeto), URLs assinadas para object storage e políticas de acesso (IAM) por serviço.

10. O que mudaria na arquitetura se o número de usuários e o volume de dados aumentassem significativamente? Escalar horizontalmente a aplicação (várias réplicas atrás de um load balancer), o que **exige** que o estado saia do container e vá para serviços compartilhados: banco gerenciado com réplicas de leitura para os resultados estruturados, object storage (que já escala praticamente sem limite) para exports/datasets/backups, cache distribuído para dados quentes, logs centralizados e CDN para servir arquivos estáticos/exports. O volume local deixa de ser suficiente para dados compartilhados — passa a servir apenas estado operacional efêmero e local a cada réplica.

7. Conclusão técnica

O laboratório evidenciou, com execução real, a distinção central do tema: **o ciclo de vida do container é independente do ciclo de vida do dado**. A camada de escrita do container é efêmera (Desafio 1), enquanto o Docker Volume persiste e pode ser compartilhado, versionado, copiado para backup e restaurado (obrigatório + Desafios 2–5). Os testes de desempenho (Desafio 7) mostraram que o custo de mover dados cresce com o tamanho e que o overhead de orquestração é relevante, e os testes de permissões (Desafio 9) demonstraram na prática o menor privilégio.

Esses achados sustentam a arquitetura proposta na Seção 5, que **não usa um único modelo de armazenamento**, e sim combina: **banco gerenciado** para resultados estruturados (integridade e concorrência), **object storage** para exports, datasets e backups (escala e custo), **logs centralizados** com retenção curta e **volumes** apenas para estado operacional efêmero. A decisão por modelo é guiada por acesso, desempenho, escalabilidade, retenção, segurança e custo — exatamente os eixos exigidos pelo roteiro.